

Joins and the fan-out trap

When you combine two tables, how do you know the total is still right?

BY THE END OF TODAY, YOU CAN

Class objectives

-
- 01 Explain why tables are split
 - 02 Run the Five Checks before a join
 - 03 Say which rows each join and UNION return
 - 04 Audit a join and fix an inflated total
 - 05 Find unmatched rows with LEFT JOIN and IS NULL
-

Warm-up

Five questions on D04, ungraded.

Suggested time • 10 minutes

PART 02

Why the data is split, and how it joins back

Yesterday's open question needs two tables at once.

Why the data is split

One customer, many orders, and why the name is stored only once.

Suggested time • 8 minutes

[labs/D05/01-Ins-Why-Tables-Split/README.md](https://github.com/DataCamp/labs/D05/01-Ins-Why-Tables-Split/README.md)

WHAT IF

If every order carried the customer's city

Lina moves from Amman to Aqaba. How many rows do you change?

order_id	status	customer_name	city
1001	completed	Omar Nasser	Irbid
1002	cancelled	Lina Haddad	Amman
1003	completed	Omar Nasser	Irbid
1004	completed	Omar Nasser	Irbid
1005	completed	Lina Haddad	Amman

One customer, many orders

C-118 has two orders. Each order has exactly one customer.

Primary key

Foreign key

customer_id	customer_name	1	many	order_id	customer_id	status
C-118	Lina Haddad			1001	C-204	completed
C-204	Omar Nasser			1002	C-118	cancelled
C-377	Rana Khouiry			1003	C-204	completed
C-401	Omar Nasser			1004	C-401	completed
C-455	Sami Odeh			1005	C-118	completed

DEFINITION

A join puts the columns of two tables side by side for one query, matching rows on a key; neither table changes.

The Five Checks

Run them out loud before every join this week, and in Assignment 1.

Suggested time • 7 minutes

[labs/D05/01-Ins-Why-Tables-Split/README.md](https://github.com/DataCamp/labs/blob/main/D05/01-Ins-Why-Tables-Split/README.md)

The Five Checks before a join

-
- 01 What is one row in each table?
 - 02 Which key joins them?
 - 03 Is the key unique on the “one” side?
 - 04 How many rows should come out?
 - 05 What happens to rows with no match?
-

Is the key unique on the one side?

Equal counts mean every customer_id appears once in customers.

```
SELECT COUNT(*) AS customer_rows,  
       COUNT(DISTINCT customer_id) AS different_ids  
FROM D03.customers;
```

→ 5 · 5: unique

INNER JOIN and LEFT JOIN on D03

Predict 5 rows first. Checkpoint: C-377 and C-455 appear only in the LEFT JOIN.

Suggested time • 15 minutes

[labs/D05/02-Evr-Inner-and-Left/README.md](#)

AS o gives the table a short name

Both tables have customer_id, so **o.** and **c.** say which one.

```
SELECT o.order_id, o.status, c.customer_name, c.city
FROM D03.orders AS o
INNER JOIN D03.customers AS c
    ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

INNER JOIN keeps the rows that match

ON names the key on each side. Every order finds one customer.

```
SELECT o.order_id, o.status, c.customer_name, c.city
FROM D03.orders AS o
INNER JOIN D03.customers AS c
ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

→ 5 rows

customers LEFT JOIN orders

LEFT JOIN keeps every customer

7 rows. C-377 and C-455 have no orders, so their order columns are NULL.

customer_id	customer_name	order_id	status
C-118	Lina Haddad	1002	cancelled
C-118	Lina Haddad	1005	completed
C-204	Omar Nasser	1001	completed
C-204	Omar Nasser	1003	completed
C-377	Rana Khoury	NULL	NULL
C-401	Omar Nasser	1004	completed
C-455	Sami Odeh	NULL	NULL

HOW MANY ROWS?

Every match is one line

Each order points to one customer. Draw a line for each match, then count the lines.

customers

C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

orders

1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118

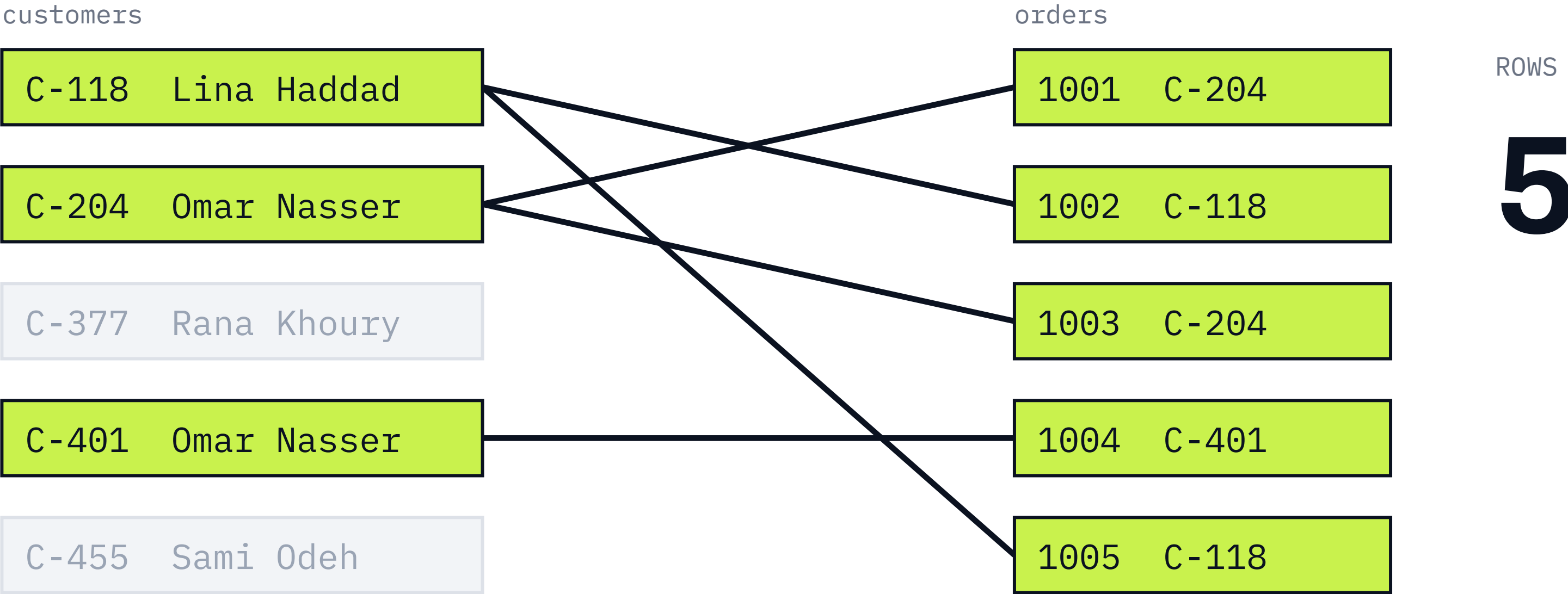
ROWS



HOW MANY ROWS?

INNER JOIN returns one row per line

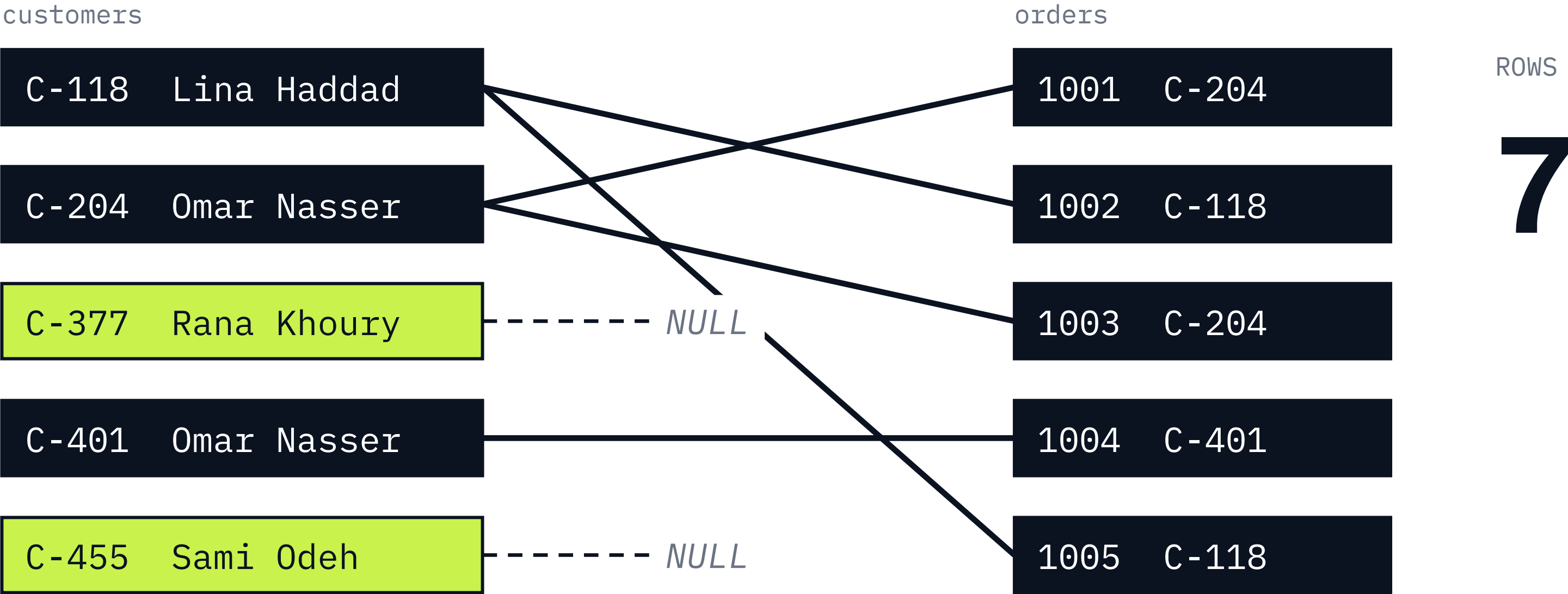
5 lines, 5 rows. C-377 and C-455 have no line, so INNER JOIN leaves them out.



HOW MANY ROWS?

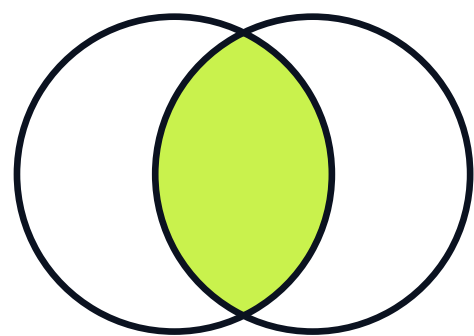
LEFT JOIN adds the customers with no line

Each left row with no line comes back once, with NULL for the order: $5 + 2 = 7$.



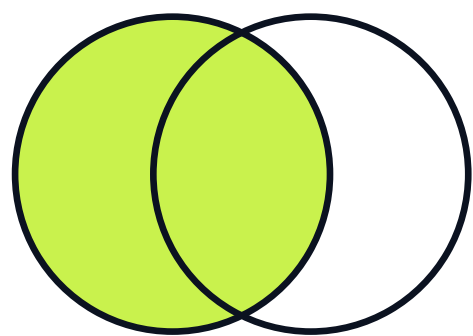
Two joins now, the rest after the audit

INNER and LEFT JOIN are the two you will write most. RIGHT, FULL, UNION and CROSS JOIN come at 4.5 to 4.7.



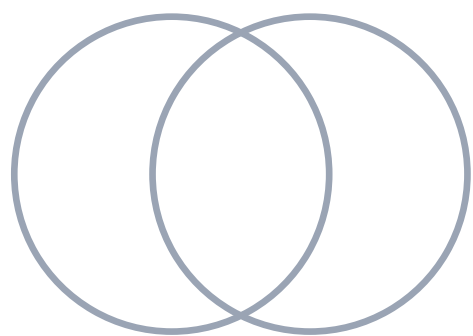
INNER JOIN

now



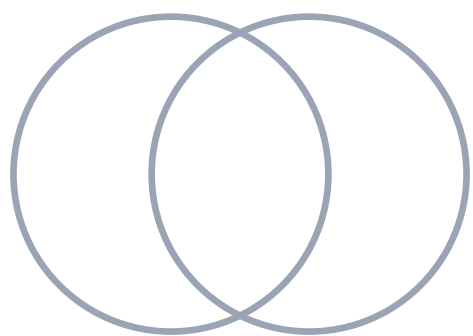
LEFT JOIN

now



RIGHT JOIN

4.5



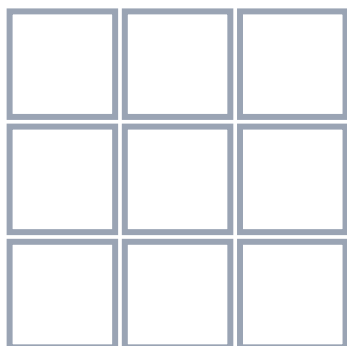
FULL OUTER

4.5



UNION

4.6



CROSS JOIN

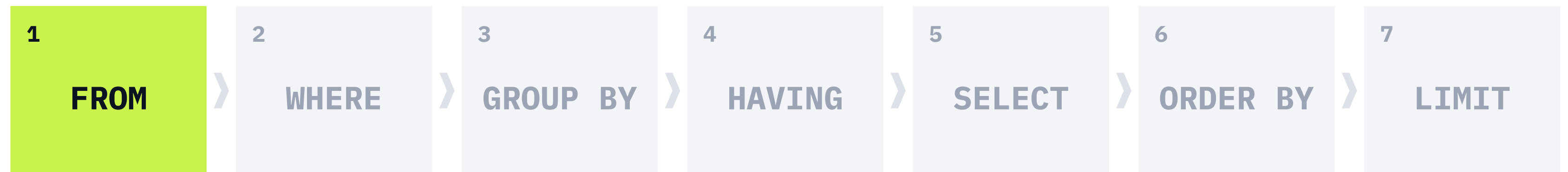
4.7

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



JOIN is part of **FROM**: the joined rows exist before WHERE runs.

Audit every join

Rows, distinct keys and the total, before and after.

How many completed orders?

The same question, before and after a join.

Suggested time • 8 minutes

[labs/D05/03-Evr-Count-Completed-Orders/README.md](#)

How many completed orders?

On orders

4

One row is one order

After joining order_items

10

One row is one product on one order

COUNT(DISTINCT o.order_id) after the join: 4 again.

Join Nakheel orders to customers

3,000 orders. Count them again after INNER JOIN and after LEFT JOIN.

Suggested time • 10 minutes

[labs/D05/04-Evr-Orders-to-Customers/README.md](https://github.com/DataCamp/labs/blob/main/D05/04-Evr-Orders-to-Customers/README.md)

INNER JOIN drops one order

orders

3,000

INNER JOIN customers

2,999

LEFT JOIN customers

3,000

Primary key

Foreign key

order_id	customer_id	order_date	status	order_total
101664	C-0999	2025-11-14	completed	938.00

C-0999 is not in customers.

STUDENT DO

Activity: Audit the join

In this activity, you will join Nakheel orders to their order lines and find which number the join changed.

Suggested time

15 minutes

[labs/D05/05-Stu-Join-Audit/README.md](#)

Activity: Audit the join

INSTRUCTIONS

1. On orders: rows, different orders, SUM(order_total).
 2. The same three after orders INNER JOIN order_items.
 3. After the join, sum qty * price instead.
- First, the Five Checks: which is the “one” side?

HINT

order_total is stored once per order. How often does it appear after the join?

BONUS

Which order had its total copied onto the most rows?

Time's up. Let's review.

Audit the join

Solved: `labs/D05/05-Stu-Join-Audit/Solved/join_audit.sql`

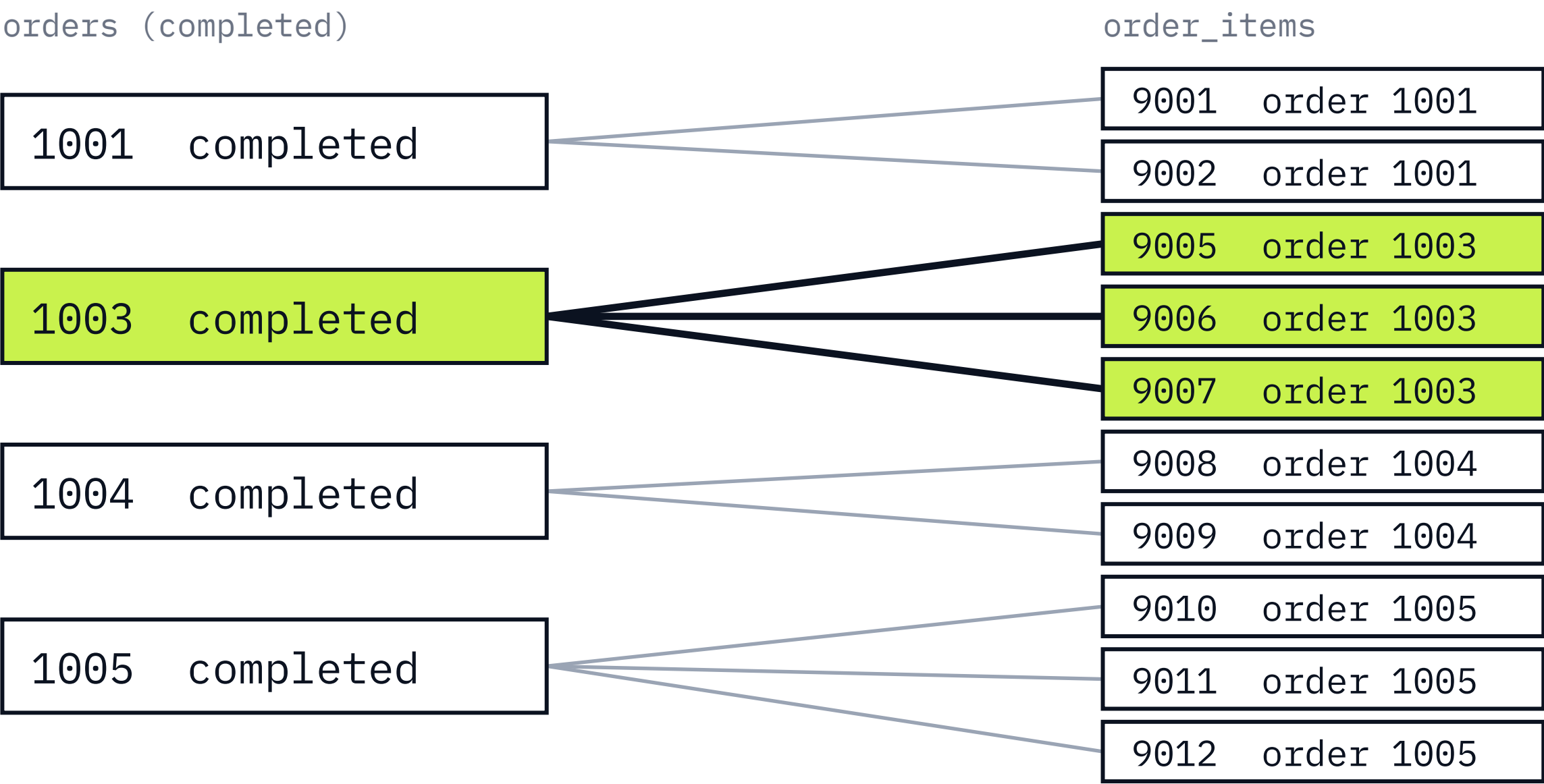
Which number did the join change?

SUM(i.qty * i.price) after the join: 340,370.00. The money is all there, summed at the wrong grain.

	Rows	Different orders	SUM(order_total)
orders	3,000	3,000	340,370.00
orders JOIN order_items	7,516	3,000	1,067,249.00

Order 1003 has three lines

4 completed orders, 10 lines. Order 1003 and its status come back three times.



ROWS

4 → 10

orders → joined rows
1003 appears 3 times

DEFINITION

Fan-out is what happens when a join to the “many” side copies each “one”-side row, and every amount stored on it.

Fix it by choosing the grain

Sum each amount at the table where it lives.

Sum each amount where it lives

Two fixes for the inflated total, and one rule.

Suggested time • 10 minutes

[labs/D05/06-Ins-Sum-Where-It-Lives/README.md](#)

INFLATED

order_total, summed after the join

order_total lives on orders. The join copied it onto every line.

```
SELECT o.channel,  
       ROUND(SUM(o.order_total), 2) AS revenue_inflated  
FROM nakheel.orders AS o  
INNER JOIN nakheel.order_items AS i  
  ON o.order_id = i.order_id  
WHERE o.status = 'completed'  
GROUP BY o.channel  
ORDER BY o.channel;
```

→ **online 351,733.50 · store 637,238.00**

FIX 1

Sum the amount stored on each line

qty * price lives on order_items, the grain of the joined rows.

```
SELECT o.channel,  
       ROUND(SUM(i.qty * i.price), 2) AS revenue  
FROM nakheel.orders AS o  
INNER JOIN nakheel.order_items AS i  
  ON o.order_id = i.order_id  
WHERE o.status = 'completed'  
GROUP BY o.channel  
ORDER BY o.channel;
```

→ **online 110,685.50 · store 205,338.00**

Sum an amount at the table where it lives, or
prove the join did not change that table's grain.

COUNT(*) against COUNT(column) after a LEFT JOIN

Why a customer with no orders counts as 1.

Suggested time • 10 minutes

[labs/D05/07-Evr-Count-After-Left-Join/README.md](#)

COUNT(*) counts the empty row too

COUNT(o.order_id) skips the NULL that the LEFT JOIN added.

```
SELECT c.customer_id,  
       COUNT(*) AS rows_counted,  
       COUNT(o.order_id) AS orders  
FROM nakheel.customers AS c  
LEFT JOIN nakheel.orders AS o  
  ON c.customer_id = o.customer_id  
GROUP BY c.customer_id  
ORDER BY orders, c.customer_id;
```

→ **C-1013: 1 row counted, 0 orders**

Activity: Never ordered, never sold

In this activity, you will find the customers who never ordered and the products never sold, with LEFT JOIN and IS NULL.

Suggested time

8 minutes

[labs/D05/08-Stu-Never-Ordered/README.md](#)

Activity: Never ordered, never sold

INSTRUCTIONS

1. Which customers signed up but never placed an order?
 2. Which products have never sold?
- Before each: which table goes on the left?

HINT

After a LEFT JOIN, test the right-hand table's key: `o.order_id IS NULL`.

BONUS

How many never-ordered customers are in each city, as typed?

Time's up. Let's review.

Never ordered, never sold

Solved: `labs/D05/08-Stu-Never-Ordered/Solved/never_ordered.sql`

The join family: RIGHT, FULL and the no-match joins

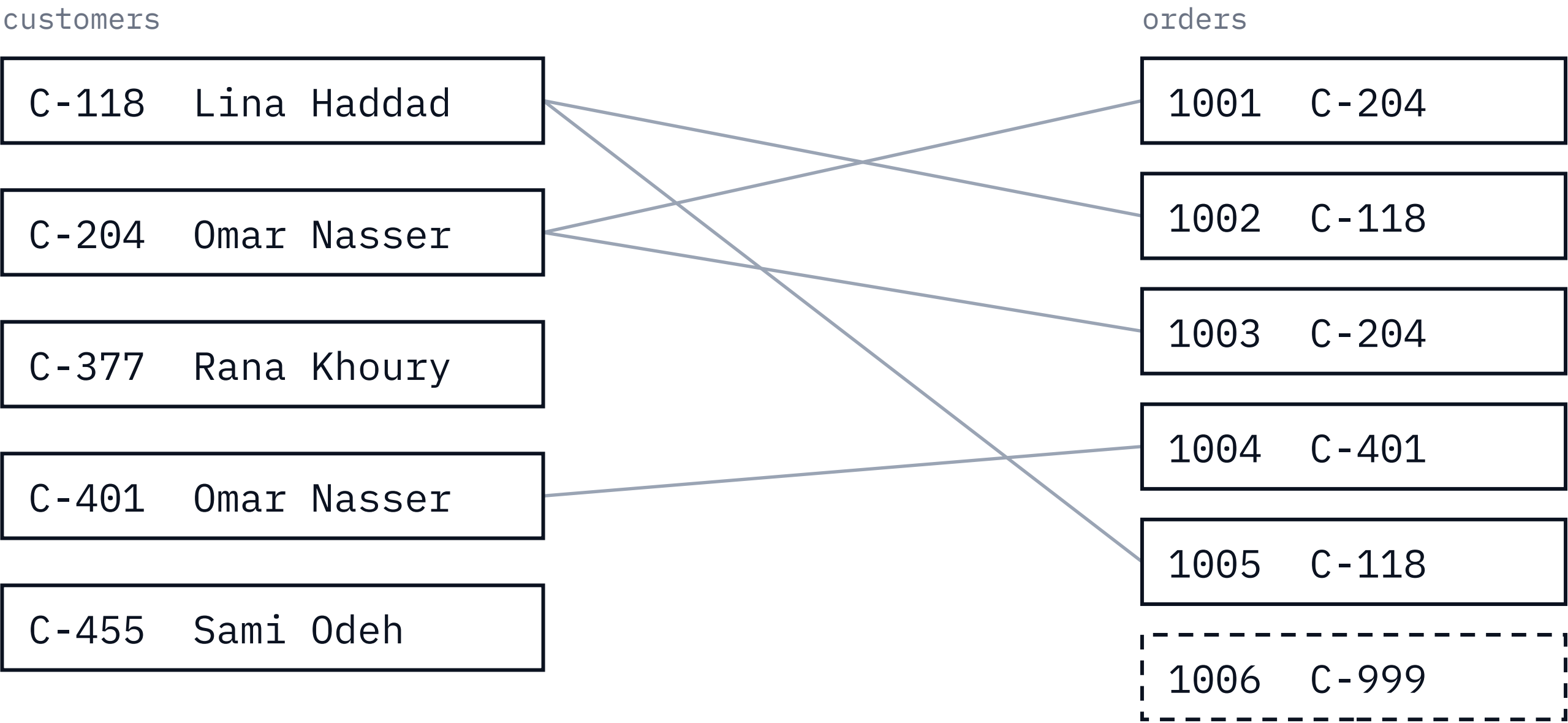
Which rows each join keeps, drawn two ways. You will read these more often than you write them.

Suggested time • 7 minutes

[labs/D05/09-Ins-Right-Full-Subquery/README.md](https://github.com/DataCamp/labs/D05/09-Ins-Right-Full-Subquery/README.md)

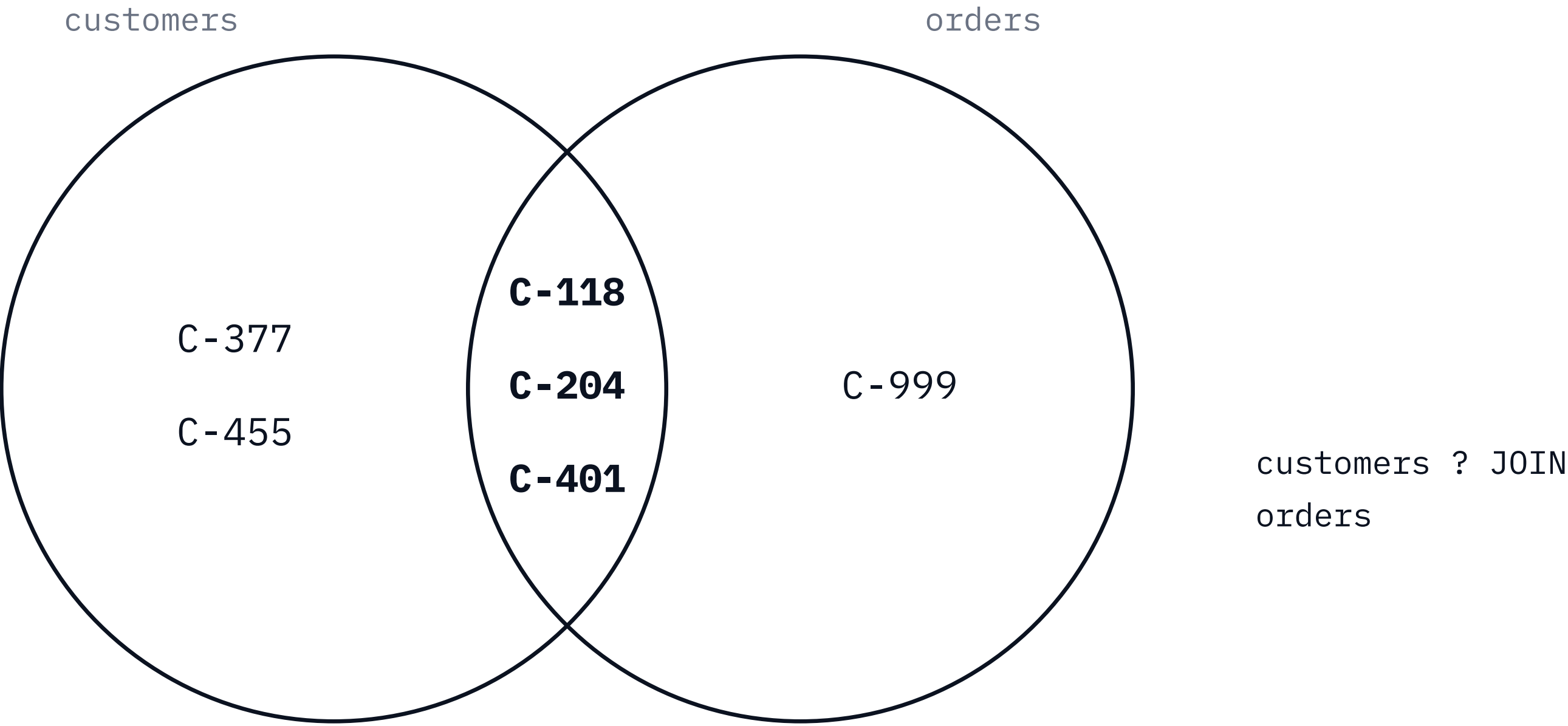
Add one order with no customer

Order 1006 points to C-999, who is not in customers. Nakheel has a real one: 101664.



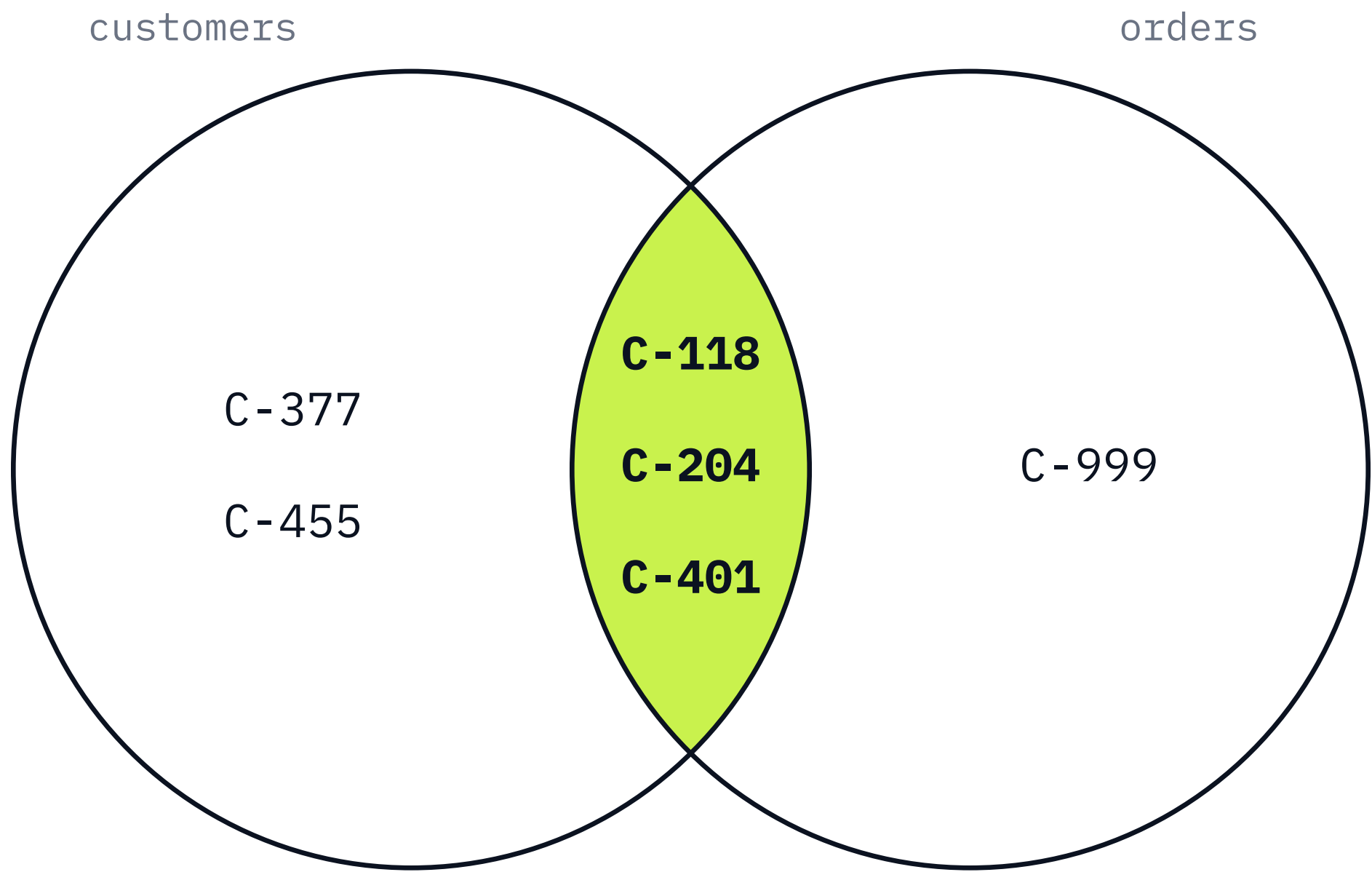
Which keys sit on each side

Each label is a customer_id value, not a row. The overlap holds the keys found in both tables.



INNER JOIN keeps the overlap

Only keys found in both tables come back.



ROWS · D03 + 1006

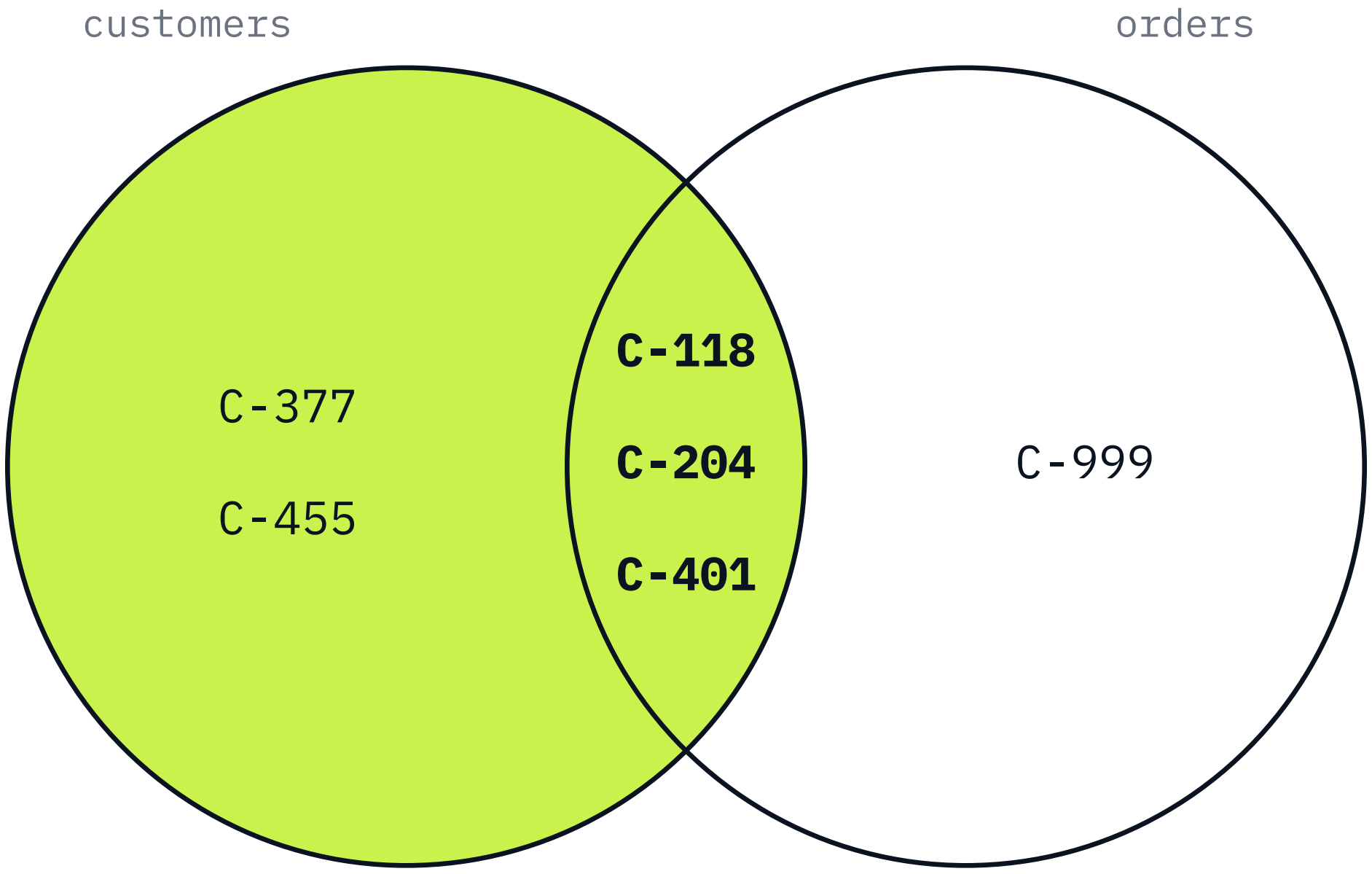
5

Nakheel: 2,999

customers
INNER JOIN orders

LEFT JOIN keeps the whole left circle

Every customer, with NULL where there is no order.



ROWS · D03 + 1006

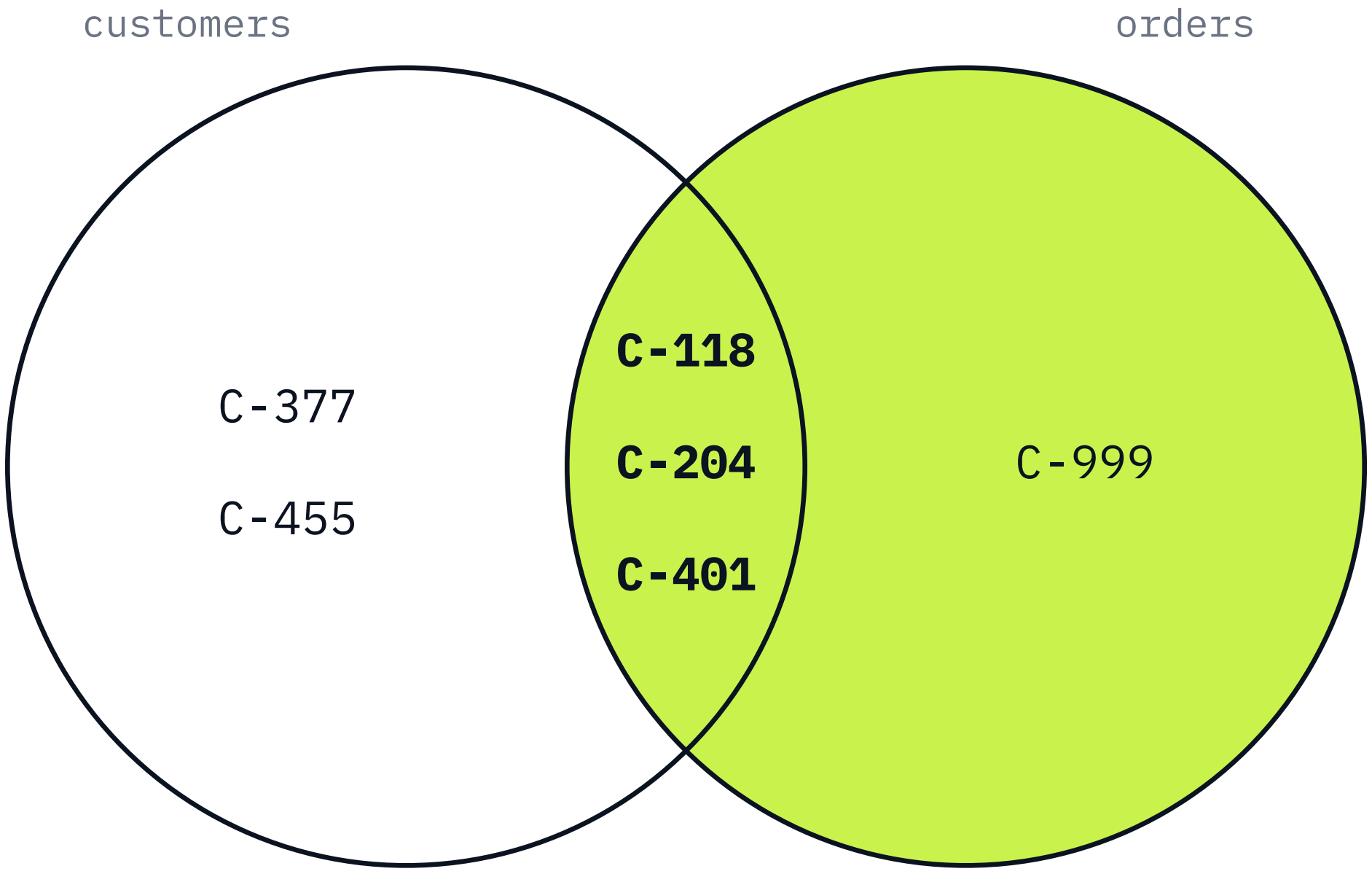
7

Nakheel: 3,038

customers
LEFT JOIN orders

RIGHT JOIN keeps the whole right circle

Every order, with NULL where there is no customer. Swap the tables and it is a LEFT JOIN.



ROWS · D03 + 1006

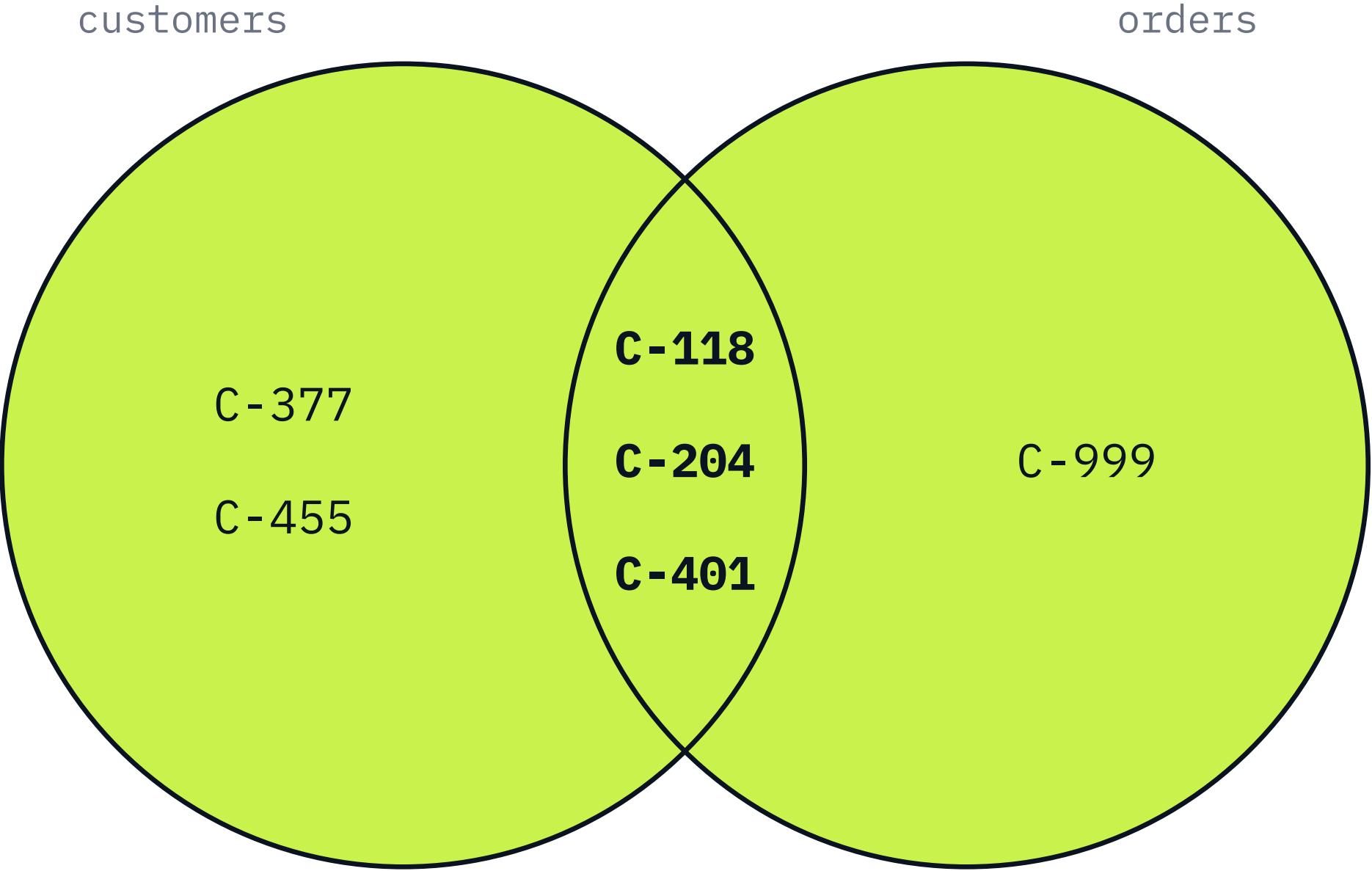
6

Nakheel: 3,000

customers
RIGHT JOIN orders

FULL OUTER JOIN keeps both circles

Every row from each side, matched where it can be.



ROWS · D03 + 1006

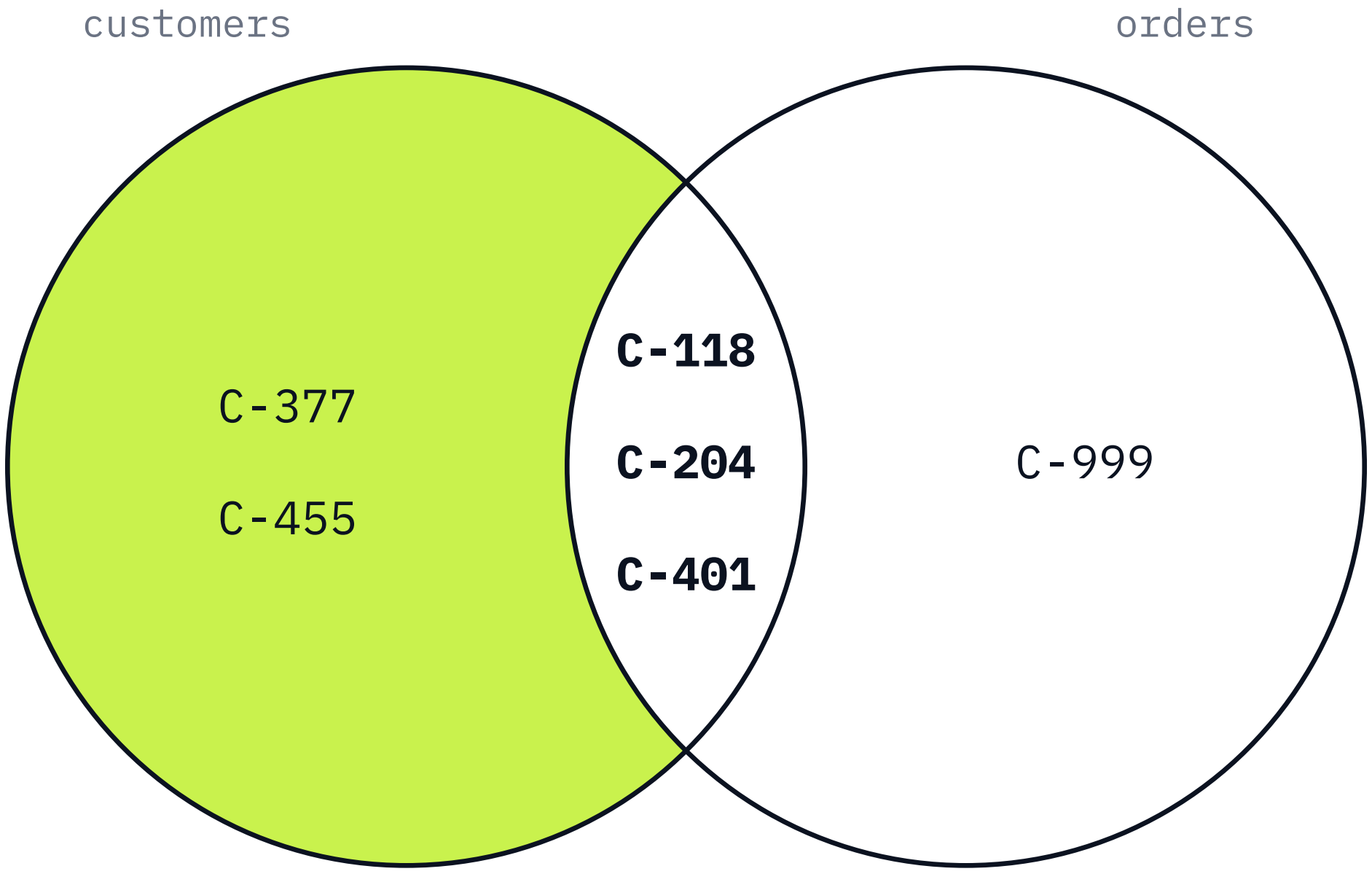
8

Nakheel: 3,039

customers
FULL OUTER JOIN
orders

No match: LEFT JOIN, then IS NULL

The left crescent is the never-ordered list from 4.3. Power Query calls it a left anti join.



ROWS · D03 + 1006

2

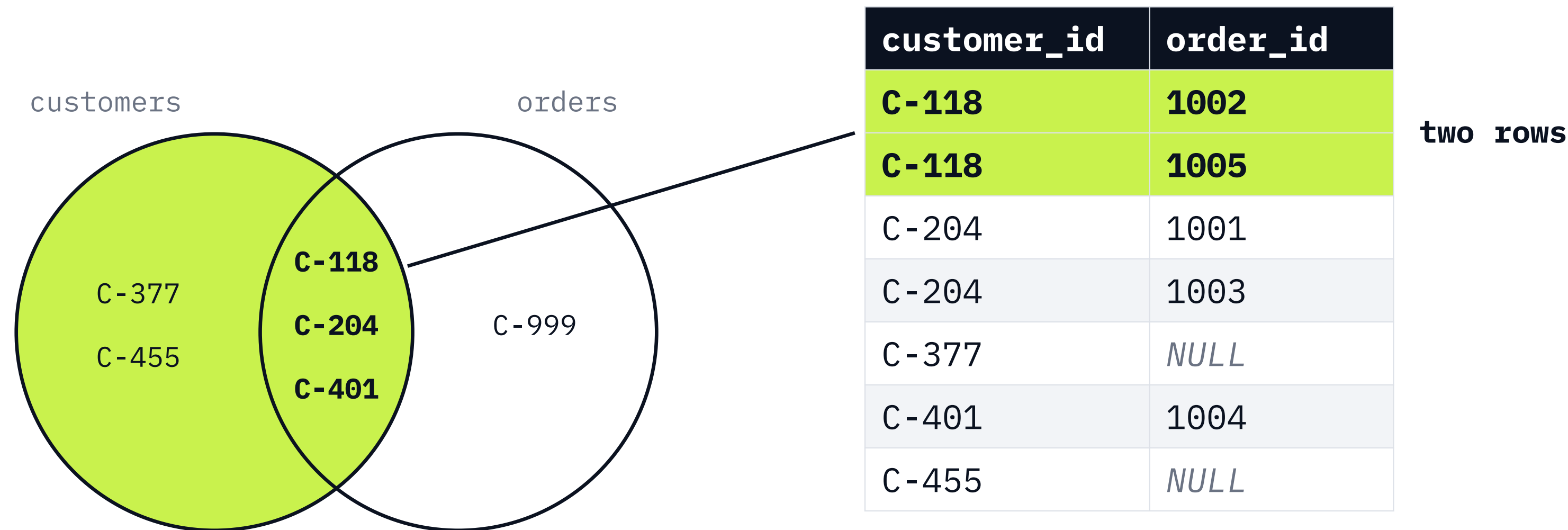
Nakheel: 39

```
LEFT JOIN orders
WHERE o.order_id
IS NULL
```

WHERE THE CIRCLES STOP

The circles hide the copies

C-118 is one key in the circles but two rows after the LEFT JOIN. Circles show which keys survive; count rows with the lines.



Same two tables, four results

The five matched rows are in every result. Each join adds a different set of unmatched rows, lit.

INNER • 5 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004

LEFT • 7 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004
C-377	NULL
C-455	NULL

RIGHT • 6 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004
NULL	1006

FULL • 8 rows

customer	order
C-118	1002
C-118	1005
C-204	1001
C-204	1003
C-401	1004
C-377	NULL
C-455	NULL
NULL	1006

UNION ALL and UNION DISTINCT

A join goes across; UNION goes down. One D03 example with both kinds of repeat.

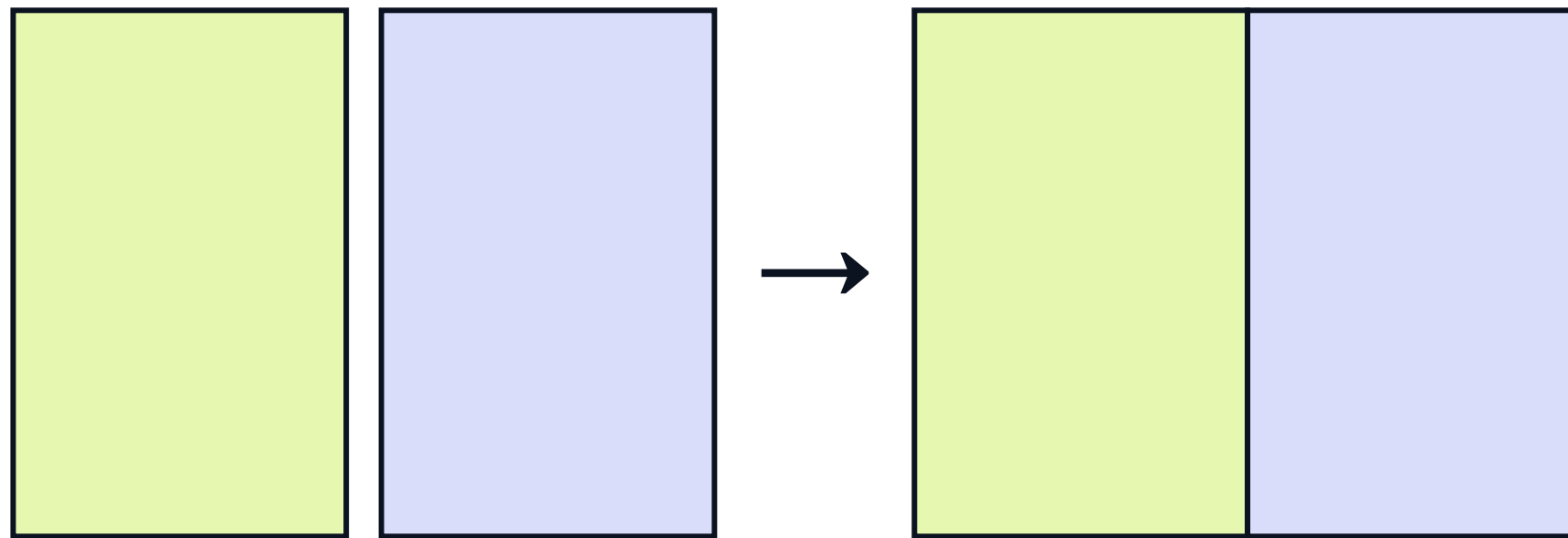
Suggested time • 6 minutes

[labs/D05/09-Ins-Right-Full-Subquery/README.md](#)

JOIN adds columns. UNION adds rows.

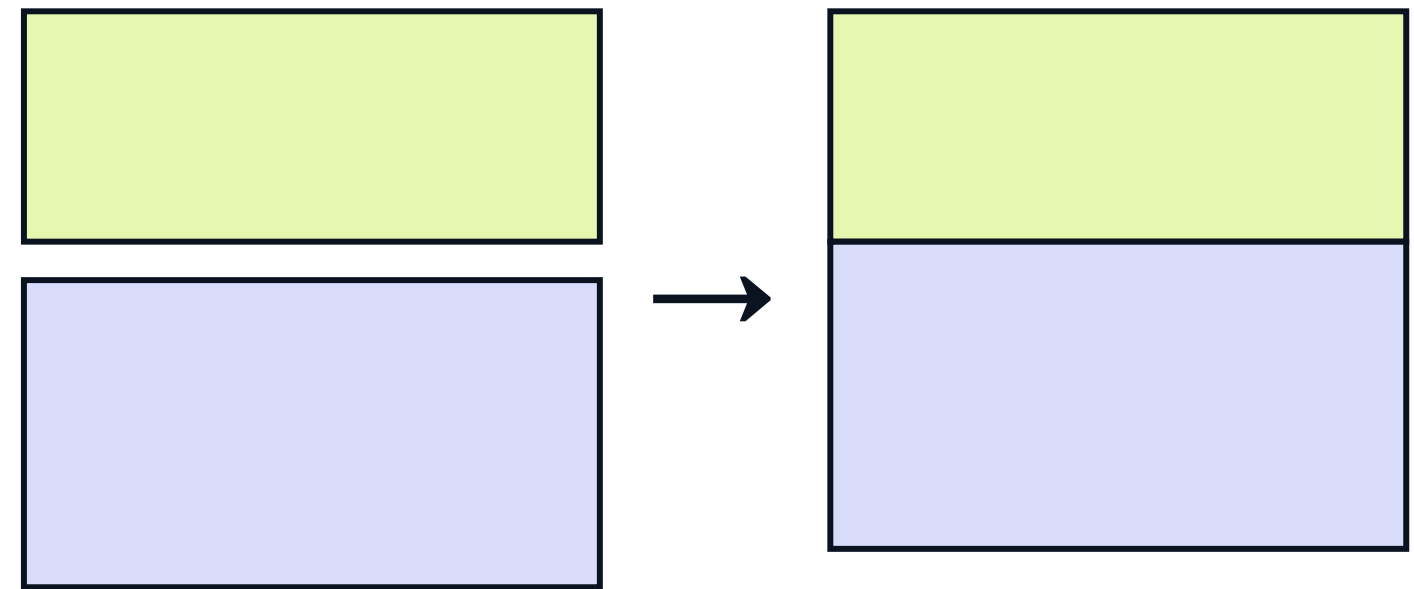
A join puts two tables side by side on a key. UNION stacks one result under another.

JOIN • goes across



more columns • rows matched on a key

UNION • goes down



more rows • same columns, no key

DEFINITION

UNION stacks the rows of two queries into one result. Both return the same number of columns, in the same order, with matching types.

Two lists of customer IDs

Q1: customers in Amman. Q2: the customer on each completed order.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118

UNION ALL keeps every row

Q2's rows go under Q1's. The count is always rows in Q1 + rows in Q2.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118



UNION ALL

customer_id
C-118
C-377
C-204
C-204
C-401
C-118

ROWS

6

2 + 4: nothing removed

Two kinds of repeat

C-118 is in both queries. C-204 repeats inside Q2, once per order.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118



UNION ALL

customer_id
C-118
C-377
C-204
C-204
C-401
C-118

← C-118: in Q1 and in Q2

← C-204: twice inside Q2

← C-118 again

UNION DISTINCT keeps each row once

DISTINCT removes the repeats inside each query as well as between them.

Q1 · customers in Amman

customer_id
C-118
C-377

Q2 · completed orders

customer_id
C-204
C-204
C-401
C-118



UNION DISTINCT

customer_id
C-118
C-204
C-377
C-401

ROWS

4

each value once
sorted with ORDER BY

BigQuery makes you choose ALL or DISTINCT

Most databases read a bare UNION as DISTINCT. BigQuery stops with a syntax error.

```
SELECT customer_id FROM D03.customers  
WHERE city = 'Amman'  
UNION  
SELECT customer_id FROM D03.orders  
WHERE status = 'completed';
```

Syntax error: Expected keyword ALL or keyword DISTINCT but got keyword SELECT

Stack data with UNION ALL, then check rows out = rows in Q1 + rows in Q2. Use UNION DISTINCT when you want a list of different values.

The same stack in Power BI

Power Query’s Append keeps every row, like UNION ALL, but lines columns up by name. SQL and DAX go by position.

	Keeps repeats	Matches columns by	Names from
SQL UNION ALL	yes	position	first query
SQL UNION DISTINCT	no	position	first query
Power Query Append	yes	name	all tables
DAX UNION()	yes	position	first table

CROSS JOIN

Every row paired with every row: the grid that every join starts from.

Suggested time • 5 minutes

[labs/D05/09-Ins-Right-Full-Subquery/README.md](https://github.com/DataCamp/labs/blob/main/D05/09-Ins-Right-Full-Subquery/README.md)

DEFINITION

CROSS JOIN pairs every row of one table with every row of the other. There is no ON: rows out = rows in A $\times$ rows in B.

5 customers × 5 orders = 25 rows

Each cell is one output row: a customer paired with an order, whether the keys match or not.

	1001 C-204	1002 C-118	1003 C-204	1004 C-401	1005 C-118	ROWS
C-118						25
C-204						
C-377						
C-401						
C-455						

The matching cells are the INNER JOIN

5 cells have the same customer_id on both sides. Every join keeps part of this grid.

	1001 C-204	1002 C-118	1003 C-204	1004 C-401	1005 C-118	ROWS
C-118						5
C-204						
C-377						
C-401						
C-455						

LEFT JOIN keeps the rows with no cell

C-377 and C-455 have no lit cell, so LEFT JOIN keeps each of them once, with NULL.

	1001 C-204	1002 C-118	1003 C-204	1004 C-401	1005 C-118		ROWS
C-118							7
C-204							
C-377						+1 row, NULL	
C-401							
C-455						+1 row, NULL	

At Nakheel, the grid has 1.5 million rows

customers	× orders	= CROSS JOIN rows
500	3,000	1,500,000

A join on a column that repeats on both sides, such as a name or a city, grows toward this. Check 3 of the Five Checks catches it before you run.

Every customer against every category

CROSS JOIN gives every pair a row, even with no sale. Translate: which customers have never bought Outerwear?

	Shoes	Accessories	Outerwear	ROWS
C-118 Lina Haddad	bought	bought	0	15
C-204 Omar Nasser	bought	bought	0	
C-377 Rana Khoury	0	0	0	
C-401 Omar Nasser	0	bought	bought	
C-455 Sami Odeh	0	0	0	

9 pairs: no sale
4: no Outerwear

D01's answer, in SQL

The category revenue you worked out by hand on the first day.

D01's answer, in SQL

Accessories, 166.00, worked out by hand on D01. Now as a query.

Suggested time • 20 minutes

[labs/D05/10-Evr-D01-Category-Revenue/README.md](#)

Two joins, nothing changed

12 rows and 487.00 before and after both joins: each line finds one order and one product.

```
SELECT p.category,  
       SUM(i.qty * i.price) AS revenue  
FROM D03.order_items AS i  
INNER JOIN D03.orders AS o ON i.order_id = o.order_id  
INNER JOIN D03.products AS p ON i.product_id = p.product_id  
WHERE o.status = 'completed'  
GROUP BY p.category  
ORDER BY revenue DESC;
```

Completed orders, by category

The same as D01. Without WHERE, Shoes wins with 195.00: the cancelled order again.

```
SELECT p.category,  
       SUM(i.qty * i.price) AS revenue  
FROM D03.order_items AS i  
INNER JOIN D03.orders AS o ON i.order_id = o.order_id  
INNER JOIN D03.products AS p ON i.product_id = p.product_id  
WHERE o.status = 'completed'  
GROUP BY p.category  
ORDER BY revenue DESC;  
  
→ Accessories 166.00 · Shoes 136.50 · Outerwear 84.00
```

Break

 Back at 7:00 PM

After the break: the lab and the Week 2 recap. Ask in your group channel.

Lab and Week 2 recap

Revenue by category, then films, then the recap.

Launch the lab

Revenue by category on Nakheel, then a film database with a table in the middle.

Suggested time • 10 minutes

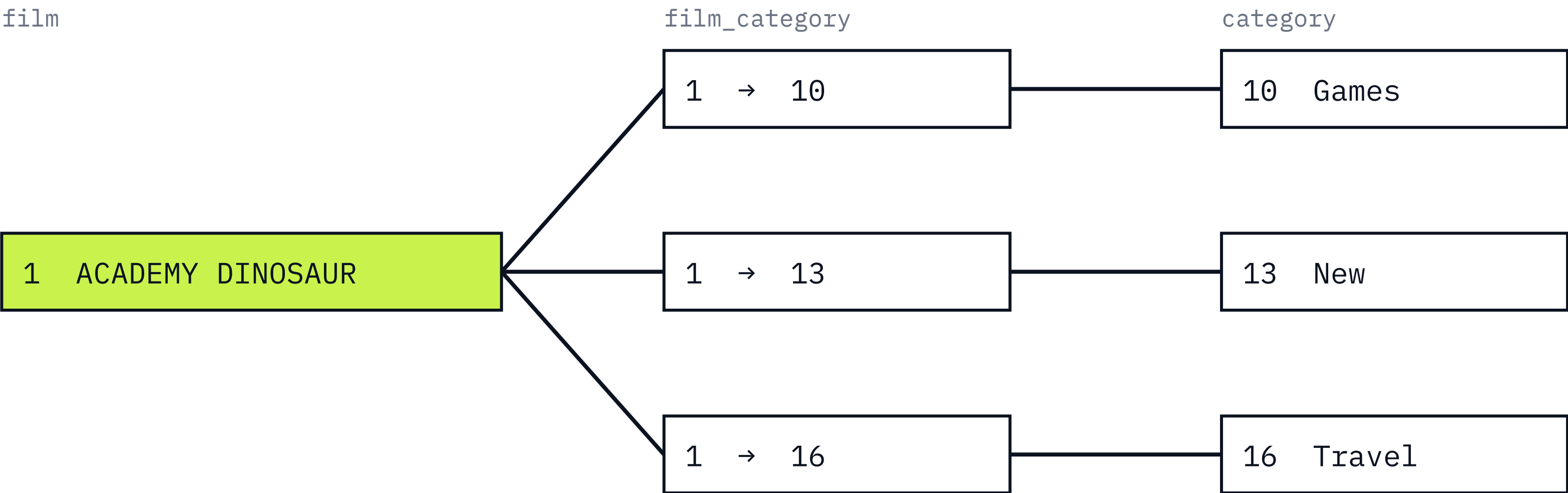
[labs/D05/11-Lab-Category-Revenue/README.md](#)

DEFINITION

A bridge table sits between two tables and holds one row per link: in Pagila, one row per film per category.

One film, three categories

film_category holds one row per film per category, so film 1 comes back three times.



Lab: Revenue by category, with a join audit

In this lab, you will answer D04's open question with an audit after every join, then use the same method on films.

Suggested time

35 minutes

[labs/D05/11-Lab-Category-Revenue/README.md](#)

Lab: Revenue by category, with a join audit

CHECKPOINTS

C1 Revenue by category from order_items and products, audited before and after.

C2 Add orders, keep completed ones, check the total against orders alone.

C3 Pagila: films per category. Why 2,367 rows, not 1,000?

HINT

Write the joins and check the audit before adding WHERE and GROUP BY.

STRETCH

The same join on thelook_ecommerce, after a key-uniqueness check.

Week 2 Mash recap

Six questions on the week, ungraded.

Suggested time • 10 minutes

Join reference card

Customers written first. Counts from the course data. UNION ALL is Power Query's Append.

Join	Keeps	D03	Nakheel	Power Query
INNER JOIN	rows with a match on both sides	5	2,999	Inner
LEFT JOIN	every left row, NULL if no match	7	3,038	Left outer
RIGHT JOIN	every right row, NULL if no match	5	3,000	Right outer
FULL OUTER JOIN	every row from both sides	7	3,039	Full outer
LEFT JOIN + IS NULL	left rows with no match	2	39	Left anti
RIGHT JOIN + IS NULL	right rows with no match	0	1	Right anti
CROSS JOIN	every pair, no ON	25	1,500,000	(none)

CLOSE

What you can do now

- Explain why tables are split
- Run the Five Checks
- Say which rows each join and UNION return
- Audit rows and totals after a join

NEXT

Keep your three .sql files for Assignment 1. D06: CTEs.